智能车辆工程赛道说明文档

1 项目简介

1.1 任务背景

随着自动驾驶技术的快速发展，对于在复杂城市环境中实现安全和高效的自动驾驶系统的需求不断增长。真实世界中，驾驶情境具有显著的多样性，涉及到各种复杂的交通状况、道路类型以及不同的昼夜、天气条件。本次比赛旨在通过目标检测和分类技术，实现对多样性场景的精准感知。

1.2 项目意义

该项目着眼于基于视觉深度学习的自动驾驶场景，旨在对车载摄像头采集的视频数据进行道路场景解析，为自动驾驶提供一种解决思路。利用 YOLO 系列模型 PP_YOLOE 完成车辆检测实现一种高效高精度的道路场景解析方式,从而实现真正意义上的自动驾驶，减少交通事故的发生，保障车主的人身安全。

1.3 方法概述

在行车检测方面，现有检测模型可以实现多种类型的车辆检测，然而，一方面，检测模型在速度和精度上存在矛盾，对于精度较高的模型，如两阶段检测网络 Faster R-CNN，其 FPS 较低，无法满足实时检测，因此其商用价值受到很大限制。另一方面，对于道路场景的目标检测，许多数据集会对场景中很多类型的目标进行标注。然而，经过我们的实践和观察，使用这种数据集训练模型并不能带来很好的效果。由于目标类别本身存在多样性，例如各式各样的货车，电瓶车，亦或是各式各样的路标和交通灯等，这会对模型的学习造成混淆，最终导致模型在现实场景中对很多目标造成误判，极大的影响模型在现实中的应用。

为解决这两个问题，我们在本项目中使用飞桨开源的 YOLOE+作为检测模型，在检测速度和检测精度上实现了较好的权衡。另外，我们对大规模道路场景数据集的标签进行了重构处理，仅学习数据集中的物体类型，保证准确率和召回率，对于其它（如：车道线）类别，暂时当作背景类，后续可以再使用初始模型重新对背景类中的部分或全部目标进行训练，让不同模型各司其职，专注于不同目标来降低错误率，提高泛化能力。在此基础上进行模型的裁剪和压缩，并将模型转换为可以在普通移动端 CPU 通过 OpenVINO 推理器即可运行的 ONNX 模型。

2 数据集说明

2.1 比赛数据集介绍

比赛数据集包括训练集和验证集。其中训练集包含 7 万幅图像，其中昼间场景图像约 40000 幅，夜间场景图像约 30000 幅。验证集包含 1 万幅图像，其中包括昼间场景图像约 6000 幅，夜间场景图像约 4000 幅。其中，用于测评模型性能的测试集为非公开数据集。

2.2 标注文件说明

本次比赛主办方提供了两种标注文件，一种标注文件为 BDD100k 数据集官方标注文件，另一种标注文件将包含整个数据集标注信息的官方 JSON 标注文件拆分为与图片一一对应的 JSON 文件，以便于进行数据处理。为了处理数据的方

便，我们选择了基于第二种标注文件来完成比赛。

2.3 数据集处理

由于主办方提供的 BDD100K 数据集很多不能直接用，需要进行格式转换，所以我们将该数据集转换为 BDD100K 的目标检测部分（基于 JSON 格式文件转换成 VOC 格式文件），并生成了 train.txt、val.txt 和 labels.txt。

此外，为了保证数据集格式转换的准确性，我们编写了查看数据集数量的 Python 代码，得到结果为训练集图片总量：70000、验证集图片总量：10000。

3 技术路线

3.1 模型选择

比赛数据集包括了不同的城市道路、天气条件和地理环境，并涵盖了白昼和夜间场景，具有显著的多样性。因此，更加需要模型具有更好泛化能力的算法，以提升模型在非公开数据集的性能。

PaddleDetection 为基于飞桨的端到端目标检测套件，提供配置化的网络模块组件、十余种数据增强策略和损失函数等高阶优化支持和多种部署方案，在打通数据处理、模型开发、训练、压缩、部署全流程的基础上，提供丰富的方法。

PP-YOLOE 是一款高精度推理速度快的检测模型，包含骨干网络 CSPRepResNet、特征融合 CSPPAN、轻量级 ET-Head 和改进的动态匹配算法 TAL（Task Alignment Learning）等模块，并且根据不同的应用场景设计了一系列模型，即 s/m/l/x。此外，PP-YOLOE 还避免使用诸如可变形卷积或者 Matrix NMS 之类的特殊算子，使 PP-YOLOE 全系列模型能轻松地部署在 NVIDIA V100 这样的服务器 GPU 架构，经过模型裁剪、压缩和转换后也能较为流畅地运行在移动端 GPU 和高性能的 FPGA 开发板上。

3.2 环境配置

本项目使用的 Anaconda 虚拟环境的配置文件见项目文件夹 code/doc 目录下 environment.yaml 文件和 environment.txt，后续模型推理（主办方称作评估）即使用该环境进行。

注：requirements.txt 文件里的库文件为模型训练时的依赖项，由于我们模型训练使用的是服务器，未在本地 Anaconda（environment.yaml）进行，可能会出现 requirements.txt 文件和 environment.yaml 依赖项版本冲突的情况。

3.3 模型训练

对自动驾驶而言，除了模型识别的准确率外，还需要考虑模型的实时性。由于移动端 GPU 性能相较于服务器较低，甚至部分移动端仅有 CPU，模型的实时性更加成为我们需要重视的问题。

YOLOE+默认训练 80 轮，由于 GPU 资源有限，租借 GPU 服务器资金有限，外加训练时间原因，我们只训练了 30 轮，发现每轮的精度和 FPS 都会有所提升，30 轮结束后得到评估结果如下图所示。

模型	mAP	fps	模型大小
PP-YOLOE+_s	40.23%	17.59	2.76MB
PP-YOLOE+_m	45.8%	14.97	28.7MB
PP-YOLOE+_l	47.9%	10.88	196MB

30 轮模型训练速度参数比较

根据对三种模型进行 30 轮训练得到的结果，PP-YOLOE+_s 模型可达到 mAP(0.5:0.95):40.23%,fps:17.59，我们将使用基于 PP-YOLOE+_s 模型。

模型训练运行指令说明：`python tools/train.py -c configs/ppyoloe/ppyoloe_plus_crn_s_80e_coco.yml -o use_gpu=true`

导出最后一次训练的模型，用于下一节模型剪枝及转换的处理中。

3.4 模型处理

先通过模型可视化网站，查看上一节导出的模型，在可视化图像最下方找到我们需要剪枝的地方，并使用 paddle 官方提供的模型剪枝工具进行剪枝，使输出部分仅保留两个结点，即 1 和 0。

我们再把 paddlepaddle 模型转成 OpenVINO 推理器能使用的 ONNX 模型文件。通过 paddle2onnx 库（已安装好在配置环境中）进行模型转换，执行生成的 onnx 模型文件,将 onnx 转 xml，bin，mapping，并准备用于下一节模型推理。

经过我们的剪裁和模型转换后，我们所执行的模型文件仅有 278KB，相较于处理前的模型文件缩小了近 10 倍，达到了轻量级的效果。

3.5 模型推理

由于主办方模型评估（eval.py）的要求内容为模型推理，且学科界主流叫法也称作模型推理，故本节称此步骤为模型推理。

执行模型推理 eval.py 文件需要先进到我们提供的 Anaconda 虚拟环境 environment.yaml 中，需要保证 eval.py 文件所在目录存在 labels.txt 文件，/model 目录下存在我们所提供的 model.xml 模型文件及 bin ，mapping 文件。

模型推理 eval.py 文件的运行指令说明：`python eval.py`

附模型推理的核心算法函数：

① OpenVINO 模型推理器（class）：

```
class Predictor:
"""
OpenVINO 模型推理器
"""
def __init__(self, model_path):
ie_core = Core()
model = ie_core.read_model(model=model_path)
self.compiled_model = ie_core.compile_model(model=model,
device_name="CPU")
def get_inputs_name(self, num):
return self.compiled_model.input(num)
def get_outputs_name(self, num):
return self.compiled_model.output(num)
def predict(self, input_data):
return self.compiled_model([input_data])
def get_request(self):
return self.compiled_model.create_infer_request()
```

② 图像预处理：

```
def process_image(input_image, size):
"""输入图片与处理方法，按照 PP-Yoloe 模型要求预处理图片数据
```

```
Args:
input_image (uint8): 输入图片矩阵
size (int): 模型输入大小
Returns:
float32: 返回处理后的图片矩阵数据
"""
max_len = max(input_image.shape)
img = np.zeros([max_len,max_len,3],np.uint8)
img[0:input_image.shape[0],0:input_image.shape[1]] = input_image # 将图片放到正方形背景中
img = cv.cvtColor(img,cv.COLOR_BGR2RGB) # BGR 转 RGB
img = cv.resize(img, (size, size), cv.INTER_NEAREST) # 缩放图片
img = np.transpose(img,[2, 0, 1]) # 转换格式
img = img / 255.0 # 归一化
img = np.expand_dims(img,0) # 增加维度
return img.astype(np.float32)
```

③ 图像后处理：

```
def process_result(box_results, conf_results):
"""按照 PP-Yolove 模型输出要求，处理数据，非极大值抑制，提取预测结果
Args:box_results (float32): 预测框预测结果
conf_results (float32): 置信度预测结果
Returns:
float: 预测框
float: 分数
int: 类别
"""
conf_results = np.transpose(conf_results,[0, 2, 1]) # 转置
# 设置输出形状
box_results =box_results.reshape(8400,4)
conf_results = conf_results.reshape(8400,4)
scores = []
classes = []
boxes = []
for i in range(8400):
conf = conf_results[i,:] # 预测分数
score = np.max(conf) # 获取类别
# 筛选较小的预测类别
if score > 0.5:
classes.append(np.argmax(conf))
scores.append(score)
boxes.append(box_results[i,:])
scores = np.array(scores)
boxes = np.array(boxes)
result_box = []
result_score = []
```

```
result_class = []
# 非极大值抑制筛选重复的预测结果
if len(boxes) != 0:
# 非极大值抑制结果
indexs = tf.image.non_max_suppression(boxes,scores,len(scores),0.25,0.35)
for i, index in enumerate(indexs):
result_score.append(scores[index])
result_box.append(boxes[index,:])
result_class.append(classes[index])
# 返回结果
return np.array(result_box),np.array(result_score),np.array(result_class)
```

④ 画出预测框：

```
def draw_box(image, boxes, scores, classes, labels):
"""将预测结果绘制到图像上
Args:
image (uint8): 原图片
boxes (float32): 预测框
scores (float32): 分数
classes (int): 类别
lables (str): 标签
Returns:
uint8: 标注好的图片
"""colors = [(0, 0, 255), (0, 255, 0), (0, 0, 0), (255, 0, 0)]
scale = max(image.shape) / 640.0 # 缩放比例
if len(classes) != 0:
for i in range(len(classes)):
box = boxes[i,:]
x1 = int(box[0] * scale)
y1 = int(box[1] * scale)
x2 = int(box[2] * scale)
y2 = int(box[3] * scale)
label = labels[classes[i]]
score = scores[i]
cv.rectangle(image, (x1, y1), (x2, y2), colors[classes[i]], 2,
cv.LINE_8)
cv.putText(image,label+":"+str(score),(x1,y1-
10),cv.FONT_HERSHEY_SIMPLEX, 0.55, colors[classes[i]], 2)
return image
```

⑤ 读取标签：

```
def read_label(label_path):
with open(label_path, 'r') as f:
labels = f.read().split()
return labels
```

⑥ 模型推理：按主办方比赛要求，对/data/image 文件夹中的测试数据进行推理，

并将推理结果以赛事官方提供的比赛数据集标注文件一致的 UTF-8 格式编码.json 文件保存至/result，命名格式为“图片文件名.json”。项目结果此外，为了可视化结果，我们将处理后的图像（即画好预测框、标注好预测类别），保存在/data/result 目录下，可以方便用来对照.json 文件。

```
json_data = {
    "images": []
}

# 处理图像并生成结果
for image_file in os.listdir(image_folder):
    image_number, _ = os.path.splitext(image_file)
    image_path = os.path.join(image_folder, image_file)
    if os.path.isfile(image_path):
        image = cv.imread(image_path)
        image_processed = process_image(image, 640)

        request = predictor.get_request()
        request.infer({input_layer: image_processed})
        boxes_name = request.get_output_tensor(0).data
        conf_name = request.get_output_tensor(1).data

        boxes, scores, classes = process_result(box_results=boxes_name, conf_results=conf_name)

        # 绘制边界框并保存注释图像
        annotated_image = draw_box(image.copy(), boxes, scores, classes, labels)
        output_image_path = os.path.join(output_folder, f"{image_number}.jpg")
        cv.imwrite(output_image_path, annotated_image)

        # 创建 JSON 输出
        current_timestamp = int(time.time())  # 获取当前时间戳
        json_output = {
            "name": image_number,
            "frames": [
                {
                    "timestamp": current_timestamp,
                    "objects": []
                }
            ],
            "attributes": {
                "weather": "none",
                "scene": "none",
                "timeofday": "none"
            }
        }
```

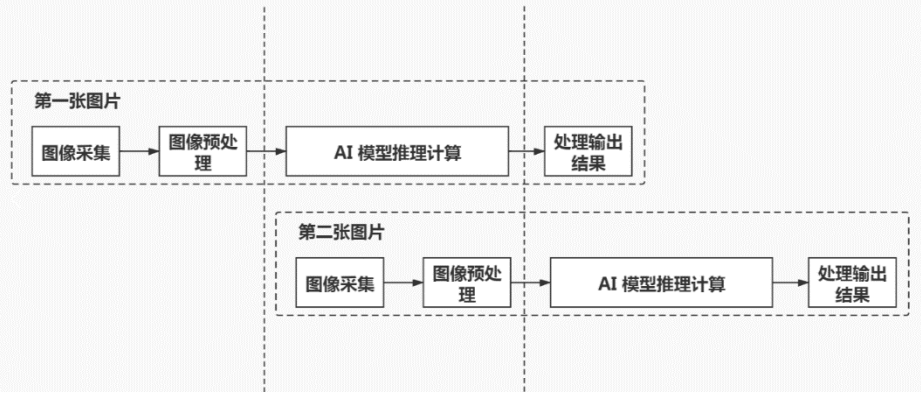
```
for i, box in enumerate(boxes):
    # 添加每个对象的识别结果到 JSON 中
    object_data = {
        "category": labels[classes[i]],
        "id": i,
        "attributes": {
            "occluded": "none",
            "truncated": "none",
            "trafficLightColor": "none"
        },
        "box2d": {
            "x1": float(box[0]),
            "y1": float(box[1]),
            "x2": float(box[2]),
            "y2": float(box[3])
        }
    }
    json_output["frames"][0]["objects"].append(object_data)

# 生成 JSON 文件路径
output_json_file = os.path.join(output_json_folder, f"{image_number}.json")

# 将 JSON 输出写入文件
with open(output_json_file, 'w', encoding='utf-8') as json_file:
    json.dump(json_output, json_file, ensure_ascii=False, indent=4)
```

3.6 实时检测系统

由于现实生活中，自动驾驶目标检测需要实时检测，因此我们在主办方要求外，自主写了一个实时性检测系统。由于使用的移动设备暂无外接相机设备，故使用笔记本自带摄像头作为本程序的接入设备。我们使用异步推理的方式，将一张图片交给模型推理的同时，程序立即获取相机下一张图片，进行相应的预处理，然后等待上一次推理结束，再把该图片交给模型推理，从而提高运行效率。



异步推理模式图

详细程序见/code 目录下的 detect.py 文件。

4 总结

我们利用 PP-YOLOE+在大规模车辆数据集 BDD100k 上进行了 30 个 epoch 的预训练，导出了 paddlepaddle 静态模型，并对 paddlepaddle 模型进行了裁剪、压缩及转换为 ONNX 模型，使其能通过 OpenVINO 在 Intel CPU 下进行模型推理，并得出了较为准确、流畅的结果。

美中不足的是，我们此项目仅能对自动驾驶汽车行驶过程中的车辆、行人、信号灯及信号牌进行识别，并不能准确感知当前环境背景，更加无法对汽车下一步的运动的进行决策。当然，本次比赛主办方也并没有要求这些。但如果想要为自动驾驶领域做出实质性的推动进展，这也要求我们在此项目基础上继续加深探索自动驾驶感知领域算法。